

Práctica 2 - Capa de Aplicación

HTTP

Introducción

1. ¿Cuál es la función de la capa de aplicación?

La capa de aplicación tiene como función principal proveer servicios de comunicación directamente a los usuarios (las personas) y a las aplicaciones de red. En esta capa es donde residen las aplicaciones de red (como la Web o el correo electrónico) y los protocolos que estas implementan.

Además, la capa de aplicación define la interfaz con el usuario (User Interface o UI) y establece aspectos críticos de la comunicación, tales como el formato de los mensajes intercambiados, la semántica de los campos de información y las reglas sobre cómo y cuándo los procesos deben enviar y responder dichos mensajes.

2. Si dos procesos deben comunicarse:

a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

Si los procesos se ejecutan en diferentes hosts (máquinas), se comunican mediante el intercambio de mensajes a través de la red de computadoras. Para lograrlo, los procesos envían y reciben mensajes hacia y desde la red utilizando una interfaz de software denominada socket, que funciona como la "puerta" entre el proceso de la capa de aplicación y la infraestructura de transporte subyacente. Para que el mensaje llegue a su destino correcto, el proceso receptor debe contar con un identificador único que está compuesto por la dirección IP de su host y un número de puerto que se asocia a su proceso específico.

b. Y si están en la misma máquina, ¿qué alternativas existen?

Si dos procesos corren en la misma máquina, se comunican utilizando mecanismos de comunicación entre procesos (IPC, por sus siglas en inglés de Inter-Process Communication), cuyas reglas y funcionamiento están definidos y administrados por el sistema operativo de dicho host. También se pueden comunicar a través de la misma red / host con el intercambio de mensajes.

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la “vida cotidiana” y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

El modelo Cliente/Servidor es una arquitectura de aplicación donde la carga de procesamiento se comparte asimétricamente. En este modelo existe un host que siempre está encendido (el servidor), el cual posee una dirección IP permanente y espera pasivamente las conexiones. Los otros hosts involucrados son los clientes, que pueden conectarse de manera intermitente y son quienes inician la comunicación contactando al servidor. En esta arquitectura, los clientes no se comunican directamente entre sí.

Ejemplo de la vida cotidiana: podríamos pensar en un alumno asistiendo a una clase y haciéndole una pregunta al profesor. El alumno (cliente) inicia la comunicación levantando la mano y realizando una solicitud (la pregunta), mientras que el profesor (servidor) está disponible para recibirla y le transmite la respuesta.

Ejemplo informático: la Web, donde un navegador (el cliente) se comunica con un servidor web solicitándole páginas y documentos.

Otros modelos de comunicación: la Arquitectura Peer-to-Peer (P2P), donde no se depende de servidores siempre activos, sino que los pares (computadoras de usuarios) se comunican directamente entre sí, pudiendo cumplir el rol de cliente y de servidor simultáneamente.

También, el Modelo Híbrido (una mezcla de Cliente/Servidor y P2P) y el antiguo Modelo Mainframe Centralizado donde clientes "tontos" solo proveen interfaz a un servidor principal que centraliza toda la carga y control.

4. Describa la funcionalidad de la entidad genérica “Agente de usuario” o “User agent”.

El agente de usuario es una entidad que actúa como interfaz entre el usuario humano y la aplicación de red. Su funcionalidad es permitir al usuario interactuar con los elementos del sistema, encargándose por detrás de enviar y recibir los mensajes por medio del socket. Por ejemplo, en la Web, el agente de usuario es el navegador web (como Chrome o Firefox) que permite visualizar e interactuar con las páginas; en el correo electrónico, es el programa (como Microsoft Outlook o Apple Mail) que le permite al usuario leer, responder, reenviar, guardar y redactar los mensajes.

5. ¿Qué son y en qué se diferencian HTML y HTTP?

HTML (HyperText Markup Language): Es el estándar y lenguaje utilizado para definir el formato y estructurar los documentos web (las páginas web) para su visualización. Es decir, HTML representa la información y el contenido en sí.

HTTP (HyperText Transfer Protocol): Es el protocolo de la capa de aplicación que rige la transferencia de esos documentos en la Web. Define cómo se estructuran y

secuencian los mensajes de solicitud (requerimientos) y de respuesta entre el cliente (navegador) y el servidor web.

Diferencia: La diferencia principal radica en que HTML es el contenido formateado que se quiere obtener, mientras que HTTP son las reglas de comunicación o el transporte utilizado para solicitar y entregar dicho contenido.

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas. (Ayuda: apartado “Formato de mensaje HTTP”, Kurose).

a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información? ¿Para qué sirven las cabeceras?

La información que nos indica si un mensaje es de requerimiento/solicitud o de respuesta se da en la primera línea de los mensajes.

- En un Mensaje de Solicitud, la primera línea se denomina línea de solicitud y tiene el formato → <Método> <URI> <Versión de HTTP>. Un ejemplo sería GET /unaDirección/página.html HTTP/1.1.
 - Método: Acción a realizar, por ejemplo GET, POST, HEAD, PUT, DELETE, etc.
 - URI (Uniform Resource Identifier): Especifica el recurso que se solicita.
 - Versión de HTTP: Indica la versión del protocolo utilizada.
- En un Mensaje de Respuesta, la primera línea se denomina línea de estado y tiene el formato → <Versión de HTTP> <Código de Estado> <Mensaje de Estado>. Un ejemplo sería HTTP/1.1 200 OK.
 - Versión de HTTP: Indica la versión del protocolo usada.
 - Código de Estado: Código numérico que indica el resultado de la solicitud.
 - Mensaje de Estado: Mensaje explicativo del estado.

Las cabeceras en HTTP proporcionan información adicional sobre la solicitud o la respuesta, como detalles sobre el cliente o servidor, tipo de contenido, codificaciones aceptadas, políticas de caché, autenticación, y más.

b. ¿Cuál es su formato? (Ayuda: <https://developer.mozilla.org/es/docs/Web/HTTP/Headers>)

Una cabecera de petición está compuesta por su nombre (no sensible a las mayúsculas) seguido de dos puntos ':', y a continuación su valor (sin saltos de línea). Los espacios en blanco a la izquierda del valor son ignorados. Se pueden agregar cabeceras propietarias personalizadas usando el prefijo 'X-'.

c. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso `/index.html`. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.

GET /index.html HTTP/1.1

Host: www.misitio.com

User-agent: curl/7.74.0

7. Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).

- **-I (o --head):** Sirve para obtener únicamente las cabeceras (headers) de la respuesta HTTP. Al usar este parámetro, curl envía una solicitud con el método HEAD en lugar del método GET tradicional. Esto resulta muy útil para examinar metadatos (como la fecha de modificación, el tipo de servidor o el tamaño del contenido) sin tener que descargar el documento completo.
- **-H (o --header):** Permite añadir líneas de cabeceras personalizadas a la solicitud HTTP que se enviará al servidor. Como se vio anteriormente en el formato de mensajes, este parámetro se usa para enviar información extra de control, como tokens de autenticación, el tipo de navegador simulado o el formato de datos deseado (ej. -H "Content-Type: application/json").
- **-X (o --request):** Se utiliza para especificar explícitamente el método HTTP con el que se realizará la petición. Aunque curl utiliza GET por defecto (o POST si se le adjuntan datos), con -X se puede forzar el uso de cualquier otro método de la capa de aplicación, como PUT, DELETE, OPTIONS, entre otros.
- **-s (o --silent):** Activa el modo silencioso. Al incluir esta bandera, curl oculta la barra de progreso, los medidores de velocidad de transferencia y los mensajes de error en la terminal. Es especialmente útil cuando se utiliza el comando dentro de scripts y se desea que la salida por pantalla sea exclusivamente la respuesta directa devuelta por el servidor.

8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (>) del comando curl a un archivo con extensión html y abrirlo con un navegador.

Se realizó un requerimiento y la respuesta es:

```

redes@debian:~$ curl www.redes.unlp.edu.ar
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>.:.Redes y Comunicaciones.:.Facultad de Inform&aacute;tica.:.UNLP.:.</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="/bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="/css/style.css" rel="stylesheet">
    <link href="/bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

    <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
    <!--[if lt IE 9]>
      <script src="/bootstrap/js/html5shiv.js"></script>
    <![endif]-->
  </head>
  <body>
    <div id="wrap">
      <div class="navbar navbar-inverse navbar-fixed-top">
        <div class="navbar-inner">
          <div class="container">
            <a class="brand" href="/index.html"><i class="icon-home icon-white"></i></a>
            <a class="brand" href="https://catedras.info.unlp.edu.ar" target=" blank">Redes y Comunicaciones</a>
            <a class="brand" href="http://www.info.unlp.edu.ar" target=" blank">Facultad de Inform&aacute;tica</a>
            <a class="brand" href="http://www.unlp.edu.ar" target="_blank">UNLP</a>
          </div>
        </div>
      </div>
    <div class="container">

```

Continúa...

Al redirigir la salida a un archivo HTML se puede abrir el archivo en un navegador para ver la página.

b. ¿Cómo funcionan los atributos href de los tags link e img en html?

Los atributos href dentro de los tags link e img funcionan como manera de importar un archivo al documento HTML actual. Establece una relación entre el documento actual y uno externo, el uso más común es en el tag link para la utilización de hojas de estilo CSS.

c. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?

No, con un solo requerimiento podemos tener acceso a la ubicación de las imágenes, pero no podemos acceder a ellas directamente. Habría que solicitarlas de manera independiente para obtener un enlace a ellas.

Por ejemplo, para obtener los estilos de la página deberíamos, primero que nada, ejecutar el comando: `curl www.redes.unlp.edu.ar`, todo esto para poder obtener la dirección de la hoja de estilos.

Y luego ejecutar el comando: `curl www.redes.unlp.edu.ar/css/style.css`, para obtener la hoja de estilos del sitio. El resultado sería:

```

redes@debian:~$ curl www.redes.unlp.edu.ar/css/style.css
/* Sticky footer styles
----- */

html,
body {
    height: 100%;
    padding-top: 30px;
    padding-bottom: 15px;
}

/* Wrapper for page content to push down footer */
#wrap {
    min-height: 100%;
    height: auto !important;
    height: 100%;
    /* Negative indent footer by it's height */
    margin: 0 auto -20px;
}

#footer {
    background-color: #f5f5f5;
}

/* Lastly, apply responsive CSS fixes as necessary */
@media (max-width: 767px) {
    #footer {
        margin-left: -20px;
        margin-right: -20px;
        padding-left: 20px;
        padding-right: 20px;
    }
}

```

d. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos CSS, dos Javascript y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente.

8 requerimientos:

- Un requerimiento para obtener el archivo HTML inicial.
- Dos requerimientos para obtener los dos archivos CSS.
- Dos requerimientos para obtener los dos archivos JavaScript.
- Tres requerimientos para obtener las tres imágenes.

9. Ejecute a continuación los siguientes comandos:

curl -v -s www.redes.unlp.edu.ar > /dev/null

curl -I -v -s www.redes.unlp.edu.ar

a. ¿Qué diferencias nota entre cada uno?

```

redes@debian:~$ curl -v -s www.redes.unlp.edu.ar > /dev/null
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sun, 29 Mar 2026 14:39:45 GMT
< Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
< Content-Length: 4898
< Content-Type: text/html
<
{ [4898 bytes data]
* Connection #0 to host www.redes.unlp.edu.ar left intact

```

```

redes@debian:~$ curl -I -v -s www.redes.unlp.edu.ar
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> HEAD / HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
HTTP/1.1 200 OK
< Date: Sun, 29 Mar 2026 14:40:08 GMT
Date: Sun, 29 Mar 2026 14:40:08 GMT
< Server: Apache/2.4.56 (Unix)
Server: Apache/2.4.56 (Unix)
< Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
< ETag: "1322-5f7457bd64f80"
ETag: "1322-5f7457bd64f80"
< Accept-Ranges: bytes
Accept-Ranges: bytes
< Content-Length: 4898
Content-Length: 4898
< Content-Type: text/html
Content-Type: text/html
<
* Connection #0 to host www.redes.unlp.edu.ar left intact

```

- Qué hacen:
 - El primer comando pide y descarga toda la página web.
 - El segundo comando pide solo los encabezados (mucho menos datos).
- Diferencias:

- En el primero, la salida se descarta completamente (> /dev/null), por lo que no se ve nada.
- En el segundo, se ven los encabezados de respuesta porque no se redirige la salida.
- Utilidades:
 - El primer comando es útil para probar la descarga completa y el tiempo de respuesta total.
 - El segundo es útil para ver rápidamente información del servidor (tipo de servidor, versión HTTP, cabeceras de seguridad, si hay redirecciones, etc.) sin descargar todo el contenido.

b. ¿Qué ocurre si en el primer comando se quita la redirección a /dev/null? ¿Por qué no es necesaria en el segundo comando?

Si se elimina la redirección en el primer comando la salida nos muestra el cuerpo de la respuesta, esto es debido a que la salida del comando como tal es solo el cuerpo, -v está mostrando el estado de la solicitud en tiempo real para hacerlo más interactivo, pero no es parte de la salida esperada.

En el segundo comando no es necesaria la redirección ya que con -I estamos forzando a que la salida sean SÓLO los encabezados, si usamos redirección en este, no saldrán duplicados los headers.

c. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?

Viajaron en el requerimiento 3 cabeceras:

- Host
- User-Agent
- Accept

Y en la respuesta 7 cabeceras:

- Date
- Server
- Last-Modified
- ETag
- Accept-Ranges
- Content-Length
- Content-Type

10. ¿Qué indica la cabecera Date?

La línea de cabecera Date indica la hora y la fecha en la que se creó la respuesta HTTP y fue enviada por el servidor. Esta no especifica la hora en que el objeto fue creado o modificado por última vez; es la hora en la que el servidor recupera el objeto de su sistema de archivos, inserta el objeto en el mensaje de respuesta y lo envía.

11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

En HTTP/1.0, por defecto no se utilizan conexiones persistentes. Esto implica que, basándose en el funcionamiento definido en las primeras versiones del protocolo, el servidor cierra la conexión TCP directamente una vez que termina de transmitir la respuesta solicitada. Por lo tanto, el cliente sabe que ha recibido el objeto completo porque el servidor finaliza la conexión al terminar de enviar los datos, o bien validando la cantidad de bytes esperados a través de la cabecera Content-Length, la cual indica el tamaño del contenido enviado.

En HTTP/1.1, el comportamiento cambia ya que las conexiones son persistentes por omisión e incorporan la capacidad de hacer pipelining (enviar múltiples solicitudes y recibir múltiples objetos sobre la misma conexión abierta). Dado que la conexión TCP no se cierra después de cada respuesta, el cliente debe utilizar otros mecanismos para saber que un objeto fue recibido completamente:

1. Leyendo la cabecera Content-Length, que le especifica el tamaño exacto del cuerpo del recurso en números decimales de octetos que le ha sido enviado.
2. A través de la cabecera Transfer-Encoding, que se utiliza para especificar la codificación de la transferencia (por ejemplo, cuando no se conoce el tamaño total por adelantado y se transfiere el objeto en fragmentos; en este caso, se puede utilizar la cabecera Trailer para incluir información adicional al final de un mensaje fragmentado).

12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

Categoría 2XX (Éxito): Indican que la petición del cliente fue recibida, comprendida y procesada con éxito.

- 200 OK: La solicitud se ha ejecutado con éxito y la información o el documento solicitado se ha devuelto en el mensaje de respuesta.
- 201 Created.
- 202 Aceptado.
- 204 No Content.
- 206 Partial Content.

Categoría 3XX (Redirección): Indican que el cliente (por ejemplo, el navegador) debe realizar una acción adicional para poder completar la solicitud, generalmente ir a una nueva URL.

- 301 Moved Permanently: El objeto solicitado ha sido movido de forma permanente a una nueva ubicación. La nueva URL se especifica en la cabecera Location: del mensaje de respuesta y el software cliente la recuperará automáticamente.
- 302 Found (Redirect temporal): Indica de forma temporal una nueva URL/URI a la que redirigirse. En este caso, el cliente no debería actualizar sus enlaces a esta nueva ubicación salvo confirmación del usuario.
- 304 Not Modified: Es una respuesta utilizada en combinación con requerimientos condicionales y cachés web. Indica a la caché que el documento no ha sido modificado desde la fecha especificada, por lo que el servidor no devuelve el cuerpo del objeto y la caché puede usar su copia almacenada, ahorrando ancho de banda.

Categoría 4XX (Errores del Cliente): Indican que hubo un problema o error en la solicitud generada por el usuario o el navegador.

- 400 Bad Request (Petición mala): Es un código de error genérico que indica que la solicitud no ha podido ser comprendida por el servidor debido a una sintaxis incorrecta.
- 401 Unauthorized: El servidor indica que el documento requiere autorización/autenticación, por lo que el cliente debe proveer credenciales válidas.
- 403 Access Forbidden (Forbidden): El acceso al recurso solicitado está prohibido para el cliente.
- 404 Not Found: El documento o recurso solicitado no se encuentra o no existe en el servidor.
- 405 Method Not Allowed: El método HTTP utilizado en la petición (por ejemplo, hacer un POST) no está permitido para ese recurso en particular.

Categoría 5XX (Errores del Servidor): Indican que el servidor es consciente de que ha ocurrido un error interno o de que es incapaz de procesar una solicitud que aparentemente es válida.

- 500 Internal Server Error: Hubo un error interno en el servidor (por ejemplo, un fallo al ejecutar un script CGI).
- 501 Method Not Implemented (Not Implemented): El método HTTP utilizado en el requerimiento no está implementado ni soportado por el servidor.

- 505 HTTP Version Not Supported: La versión del protocolo HTTP utilizada por el cliente en la solicitud no está soportada por el servidor.
- Otros códigos de error del servidor documentados incluyen 502 Puerta de enlace no válida, 503 Servicio No Disponible y 504 Gateway Timeout.

13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

a. ¿Qué información brinda la primera línea de la respuesta?

HTTP/1.1 200 OK

b. ¿Cuántos encabezados muestra la respuesta?

```
redes@debian:~/Desktop$ curl -I www.redes.unlp.edu.ar
HTTP/1.1 200 OK
Date: Sun, 29 Mar 2026 15:05:15 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 4898
Content-Type: text/html
```

c. ¿Qué servidor web está sirviendo la página?

Apache/2.4.56 (Unix)

d. ¿El acceso a la página solicitada fue exitoso o no?

Sí, esto es indicado por el código de retorno "200 OK".

e. ¿Cuándo fue la última vez que se modificó la página?

19/3/2023 19:04:46 GMT

f. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?

```
redes@debian:~/Desktop$ curl -I -H "If-Modified-Since: Sun, 19 Mar 2023 19:04:46 GMT" www.redes.unlp.edu.ar
HTTP/1.1 304 Not Modified
Date: Sun, 29 Mar 2026 15:12:46 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "1322-5f7457bd64f80"
Accept-Ranges: bytes
```

Podemos utilizar el parámetro -H para enviar un header manualmente, el cual sería If-Modified-Since: <Fecha en formato HTTP>, esto indica la fecha a partir de la cual se desea recibir la página.

Al no existir la página con una fecha de modificación desde la fecha proporcionada, el servidor responde con el código de retorno "304 Not Modified", indicando que no existe tal recurso y no proporcionará un body.

14. Utilizando curl, acceda al sitio www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.

```
redes@debian:~/Desktop$ curl www.redes.unlp.edu.ar/restringido/index.php
<h1>Acceso restringido</h1>

<p>Para acceder al contenido es necesario autenticarse. Para obtener los datos de acceso seguir las
instrucciones detalladas en www.redes.unlp.edu.ar/obtener-usuario.php</p>
redes@debian:~/Desktop$ curl www.redes.unlp.edu.ar/obtener-usuario.php
<p>Para obtener el usuario y la contraseña haga un requerimiento a esta página seteando el encabezado
'Usuario-Redes' con el valor 'obtener'</p>
```

```
redes@debian:~/Desktop$ curl -v -H "Usuario-Redes: obtener" www.redes.unlp.edu.ar/obtener-usuario.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /obtener-usuario.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Usuario-Redes: obtener
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sun, 29 Mar 2026 15:15:23 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Content-Length: 286
< Content-Type: text/html; charset=UTF-8
<
<p>Bien hecho! Los datos para ingresar son:

    Usuario: redes

    Contraseña: RYC

    Ahora vuelva a acceder a la página inicial con los datos anteriores.

    PISTA: Investigue el uso del encabezado Authorization para el método Basic. El comando base64 puede ser de ayuda!</p>
* Connection #0 to host www.redes.unlp.edu.ar left intact
```

```
redes@debian:~/Desktop$ echo -n "redes:RYC" | base64
cmVkZXN6M6U1LD
```

```
redes@debian:~/Desktop$ curl -v -H "Authorization: Basic cmVkZXN6UllD" www.redes.unlp.edu.ar/restringido/index.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/index.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXN6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 302 Found
< Date: Sun, 29 Mar 2026 15:23:30 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Location: http://www.redes.unlp.edu.ar/restringido/the-end.php
< Content-Length: 230
< Content-Type: text/html; charset=UTF-8
<
<h1>Excelente!</h1>

<p>Para terminar el ejercicio deberás agregar en la entrega los datos que se muestran en la siguiente página.</p>
<p>ACLARACIÓN: la URL de la siguiente página está contenida en esta misma respuesta.</p>

* Connection #0 to host www.redes.unlp.edu.ar left intact
```

```
redes@debian:~/Desktop$ curl -v -H "Authorization: Basic cmVkZXN6UllD" www.redes.unlp.edu.ar/restringido/the-end.php
* Trying 172.28.0.50:80...
* Connected to www.redes.unlp.edu.ar (172.28.0.50) port 80 (#0)
> GET /restringido/the-end.php HTTP/1.1
> Host: www.redes.unlp.edu.ar
> User-Agent: curl/7.74.0
> Accept: */*
> Authorization: Basic cmVkZXN6UllD
>
* Mark bundle as not supporting multiuse
< HTTP/1.1 200 OK
< Date: Sun, 29 Mar 2026 15:24:29 GMT
< Server: Apache/2.4.56 (Unix)
< X-Powered-By: PHP/7.4.33
< Content-Length: 159
< Content-Type: text/html; charset=UTF-8
<
¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2026-03-29 15:24:29
* Connection #0 to host www.redes.unlp.edu.ar left intact
```

15. Utilizando la VM, realice las siguientes pruebas:

- Ejecute el comando `'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt'` y copie la salida completa (incluyendo los dos saltos de línea del final).

```
redes@debian:~$ curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*
```

- Desde la consola ejecute el comando `telnet www.redes.unlp.edu.ar 80` y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de

hacerlo?

```
redes@debian:~$ telnet www.redes.unlp.edu.ar 80
Trying 172.28.0.50...
Connected to www.redes.unlp.edu.ar.
Escape character is '^]'.
GET /http/HTTP-1.1/ HTTP/1.0
User-Agent: curl/7.38.0
Host: www.redes.unlp.edu.ar
Accept: */*

HTTP/1.1 200 OK
Date: Thu, 09 Apr 2026 12:04:03 GMT
Server: Apache/2.4.56 (Unix)
Last-Modified: Sun, 19 Mar 2023 19:04:46 GMT
ETag: "760-5f7457bd64f80"
Accept-Ranges: bytes
Content-Length: 1888
Connection: close
Content-Type: text/html

<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="utf-8">
    <title>Protocolo HTTP: versiones</title>
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <meta name="description" content="">
    <meta name="author" content="">

    <!-- Le styles -->
    <link href="../../bootstrap/css/bootstrap.css" rel="stylesheet">
    <link href="../../css/style.css" rel="stylesheet">
    <link href="../../bootstrap/css/bootstrap-responsive.css" rel="stylesheet">

    <!-- HTML5 shim, for IE6-8 support of HTML5 elements -->
```

c. Repita el proceso anterior, pero copiando la salida del recurso /extras/prueba-http-1-1.txt. Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.

La salida es la misma pero la diferencia está en que HTTP 1.0 abre una nueva conexión TCP para cada solicitud/respuesta y la cierra inmediatamente, mientras que HTTP 1.1 utiliza conexiones persistentes (Keep-Alive) para reutilizar la misma conexión en múltiples peticiones.

Además algo que noté es que con la salida del recurso que termina en http-1-1 puedo seguir pegando varias veces el contenido y la conexión no se cierra por sí sola, si no que dura unos segundos.

16. En base a lo obtenido en el ejercicio anterior, responda:

a. ¿Qué está haciendo al ejecutar el comando telnet?

Al ejecutar telnet <servidor> 80, se está estableciendo una conexión TCP contra el puerto 80 del servidor (el puerto estándar de HTTP). Telnet actúa como un cliente TCP genérico que permite enviar y recibir texto plano, lo que nos permite escribir

manualmente una solicitud HTTP "a mano" y ver la respuesta cruda del servidor, sin que un navegador interprete o modifique nada.

b. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?

Se utilizó el método HTTP GET y se solicitó el recurso /http/HTTP-1.1/.

c. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?

La diferencia principal es el comportamiento de la conexión TCP:

- Con HTTP/1.0, la conexión se cierra inmediatamente después de recibir la respuesta. El servidor envía el recurso y corta la conexión TCP (telnet se cierra solo). Esto ocurre porque en HTTP/1.0 el comportamiento por defecto es no persistente: una conexión por cada par request/response.
- Con HTTP/1.1, la conexión queda abierta después de recibir la respuesta, esperando nuevos requests. Esto es porque HTTP/1.1 usa conexiones persistentes por defecto (el header implícito es Connection: keep-alive). La conexión se cierra recién cuando pasa un timeout o cuando se envía explícitamente Connection: close.

Otra diferencia: en HTTP/1.1 es obligatorio incluir el header Host:, ya que un mismo servidor puede alojar múltiples sitios (virtual hosting). En HTTP/1.0 esto no es requerido.

d. ¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?

HTTP/1.1 es más eficiente. Cuando una página web tiene múltiples recursos (HTML, CSS, JavaScript, imágenes), con HTTP/1.0 se necesita abrir y cerrar una conexión TCP por cada recurso, lo que implica realizar el three-way handshake cada vez, sumando latencia y overhead. Con HTTP/1.1, al ser la conexión persistente, se pueden enviar múltiples requests sobre la misma conexión TCP, ahorrando el costo de establecimiento repetido.

Sin embargo, el problema asociado es que las conexiones persistentes consumen recursos en el servidor. Cada conexión abierta ocupa memoria, sockets y file descriptors. Si hay muchos clientes manteniendo conexiones abiertas simultáneamente (incluso sin enviar requests), el servidor puede quedarse sin recursos para atender nuevos clientes. Por eso los servidores configuran timeouts para cerrar conexiones inactivas después de cierto tiempo.

17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark. Antes de iniciar considere:

- Capture los paquetes utilizando la interfaz con IP 172.28.0.1. (Menú “Capture ->Options”. Luego seleccione la interfaz correspondiente y presione Start).

- Para que el analizador de red sólo nos muestre los mensajes del protocolo http introduciremos la cadena ‘http’ (sin las comillas) en la ventana de especificación de filtros de visualización (display-filter). Si no hiciéramos esto veríamos todo el tráfico que es capaz de capturar nuestra placa de red. De los paquetes que son capturados, aquel que esté seleccionado será mostrado en forma detallada en la sección que está justo debajo. Como sólo estamos interesados en http ocultaremos toda la información que no es relevante para esta práctica (Información de trama, Ethernet, IP y TCP). Desplegar la información correspondiente al protocolo HTTP bajo la leyenda “Hypertext Transfer Protocol”.

- Para borrar la caché del navegador, deberá ir al menú “Herramientas->Borrar historial reciente”. Alternativamente puede utilizar Ctrl+F5 en el navegador para forzar la petición HTTP evitando el uso de caché del navegador.

- En caso de querer ver de forma simplificada el contenido de una comunicación http, utilice el botón derecho sobre un paquete HTTP perteneciente al flujo capturado y seleccione la opción Follow TCP Stream.

a. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección “Capa de Aplicación” llamado “Métodos HTTP”. En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario como el siguiente:



Formulario de registro con los siguientes campos:

- Nombre:
- Apellido:
- Email:
- Sexo: Masculino: ☒ Femenino: ☐
- Contraseña:
- Recibir confirmaciones por email: ☐
- Botón Enviar:

b. Analice el código HTML

c. Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.

- d. ¿Qué diferencias detectó en los mensajes enviados por el cliente?
- e. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.

19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?

20. Responder las siguientes preguntas:

a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2? (Ayuda:

<https://undertow.io/blog/2015/04/27/An-in-depth-overview-of-HTTP2.html> para HTTP/2)

b. En HTTP/1.1, ¿es correcto el siguiente requerimiento?

GET /index.php HTTP/1.1

User-Agent: curl/7.54.0

c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?

GET /index.php HTTP/1.1

Host: www.info.unlp.edu.ar

Ejercicio de Parcial

```
curl -X ?? www.redes.unlp.edu.ar/??
```

```
> HEAD /metodos/ HTTP/??
```

```
> Host: www.redes.unlp.edu.ar
```

```
> User-Agent: curl/7.54.0
```

< HTTP/?? 200 OK

< Server: nginx/1.4.6 (Ubuntu)

< Date: Wed, 31 Jan 2018 22:22:22 GMT

< Last-Modified: Sat, 20 Jan 2018 13:02:41 GMT

< Content-Type: text/html; charset=UTF-8

< Connection: close

- a. ¿Qué versión de HTTP podría estar utilizando el servidor?
- b. ¿Qué método está utilizando? Dicho método, ¿retorna el recurso completo solicitado?
- c. ¿Cuál es el recurso solicitado?
- d. ¿El método funcionó correctamente?
- e. Si la solicitud hubiera llevado un encabezado que diga: If-Modified-Since: Sat, 20 Jan 2018 13:02:41 GMT ¿Cuál habría sido la respuesta del servidor web? ¿Qué habría hecho el navegador en este caso?